SOFTWARE TECHNICAL PROPOSAL DOCUMENTATION

PROJECT: Traduce Señas

TEAM MEMBERS:
Luis Alejandro Duarte Aldana
Juan Mauricio Suaza Solorzano
Juan Camilo Fierro Esquivel

NATIONAL LEARNING SERVICE – SENA
Software Analysis and Development

2026

Table of Contents

Introduction 3
Description of the problem and the need 3
Proposed solution 3
Technical scope 4
Selected technologies 5
Alternatives evaluated 5
Proposed architecture 6
Development methodology 6
Schedule 7
Resources 7
    Human resources 7
    Technical resources 7
Budget 8
Main technical risks 8
Deliverables 8
Conclusions 9
References 10

Introduction

The technical proposal describes the software solution that the team proposes for the communication problem between deaf and hearing people, together with the chosen technologies, the architecture, the work methodology, the schedule, the resources, and the budget needed to build it.

This document is intended for the instructor and any technical evaluator who needs to understand why the solution is built the way it is and which alternatives were discarded.

Description of the problem and the need

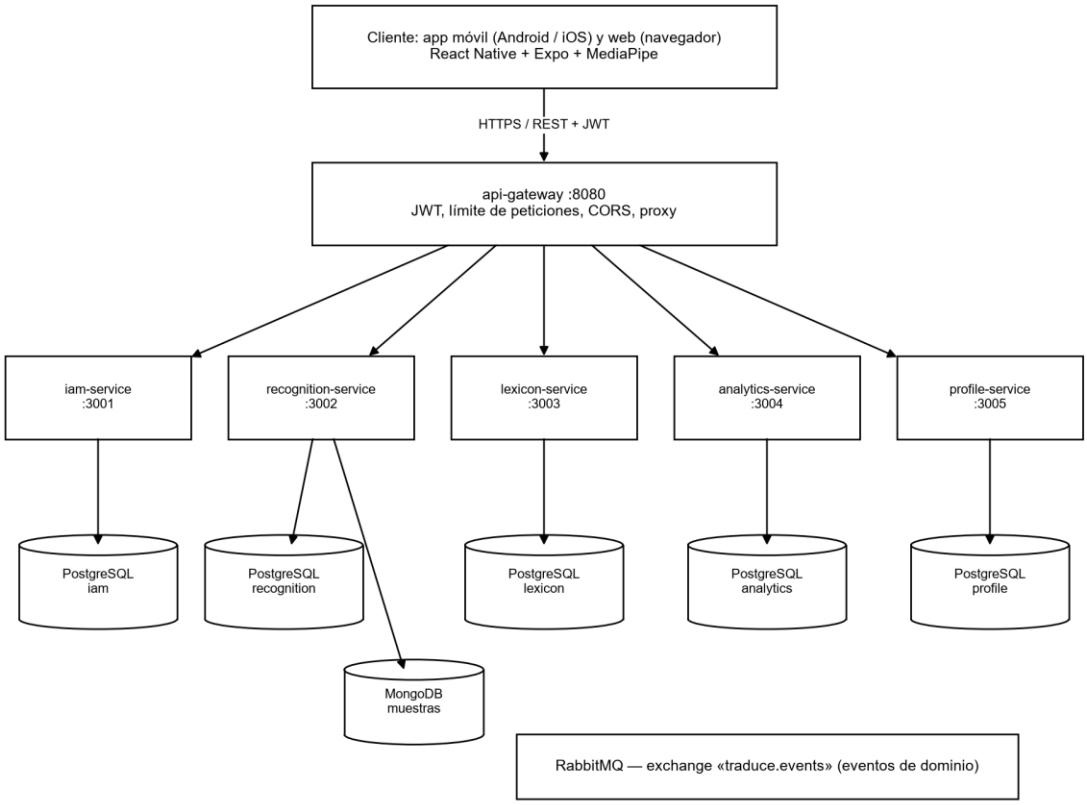
In Colombia, the number of LSC interpreters is far lower than the number of deaf people who need them (INSOR, 2020). The World Health Organization (2021) estimates that more than 1,500 million people worldwide live with some degree of hearing loss. Without an interpreter, a deaf person cannot communicate fluently at school, at the shop, or at the counter of a public agency. A tool available on any phone is needed that translates signs into text or voice immediately.

Proposed solution

A cross-platform application is proposed that uses the device's camera to recognize signs and display their translation. The solution consists of:

- **Cross-platform client** built with React Native and Expo, compiled for Android, iOS, and web (desktop) from a single codebase.
- **On-device recognition engine** with MediaPipe Hands (Zhang et al., 2020), which extracts 21 hand landmarks per frame, and a template-based classifier based on Dynamic Time Warping.
- **Microservices backend** in Node.js, Express, and TypeScript, behind an API Gateway, with one database per service.
- **Event-based communication** with RabbitMQ to keep the services decoupled.
- **Container-based deployment** with Docker and Docker Compose, reproducible on any server.

Figure 1
Overview of the proposed architecture



Note. Own elaboration.

Technical scope

Table 1
Functional modules and responsible service

Module	Functionality	Service
User management	Registration, email verification, login (also with Google and Facebook), password recovery, roles, and account deletion	iam-service
Translation	Sign recognition, text and voice, history	App + recognition-service
Training	Sample capture, sign templates, model registration	recognition-service
Lexicon and alphabet	Sign catalog, categories, alphabet with 3D models	lexicon-service
Profile	Personal data, theme, language, notifications (also weekly and monthly reports by email to administrators), achievements, and daily challenges	profile-service
Statistics	Usage events and dashboards for the administrator	analytics-service

Module	Functionality	Service
Single entry point	Routing, JWT validation, rate limiting, CORS	api-gateway

Out of scope: reverse translation (from text or voice to signs), full coverage of the LSC vocabulary, and offline use.

Selected technologies

Table 2
Technology stack and justification

Layer	Technology	Version	Justification
Client	React Native + Expo	RN 0.86 / Expo 57	A single codebase for Android, iOS, and web; the team knows JavaScript
Client	React	19.2	Common UI library for mobile and web
Client	React Navigation	6.x	Standard stack and tab navigation in React Native
Vision	MediaPipe Tasks Vision	0.10.14	Pre-trained, real-time hand detection on the device
Vision	TensorFlow.js	4.22	Execution of classification models on the client
Voice	expo-speech	57.x	Native speech synthesis for audio output
Backend	Node.js + Express	20 LTS / 5.2	Same language as the client; sufficient for the project's traffic
Backend	TypeScript	5.6+ (server) and 6.0 (client)	Static typing that reduces errors in a shared codebase
Data	PostgreSQL	16	Robust, free relational database, one per service
Data	MongoDB	7	Flexible documents for the training samples
Messaging	RabbitMQ	3.x	Event exchange between services; simple and with a web panel
Security	JWT (HS256) + bcrypt	—	Stateless authentication and hashed passwords
Infrastructure	Docker + Docker Compose	24+ / 2.20+	Reproducible environment with a single command
Version control	Git + GitHub	—	Feature branches, pull requests, and review

Alternatives evaluated

Table 3
Technical decisions and discarded alternatives

Decision	Discarded alternative	Reason
Microservices with API Gateway	Modular monolith	Allows recognition to scale separately and divides the work by domain; decision confirmed by the product leader on October 4, 2026
PostgreSQL per service	Shared MySQL	Data isolation per service, JSONB types, and more expressive constraints

Decision	Discarded alternative	Reason
RabbitMQ	Apache Kafka	Kafka is disproportionate for the project's volume; RabbitMQ is simpler to operate
On-device recognition	Sending video to the server	Lower latency, lower data consumption, and the video does not leave the phone (privacy)
Templates + DTW	Neural network trained from scratch	Works with few examples per sign and is explainable; the network remains a future improvement
Expo (React Native)	Separate native development (Kotlin and Swift)	A three-person team cannot maintain three codebases

Proposed architecture

Each microservice follows the hexagonal architecture (Cockburn, 2005): the domain and use cases do not depend on Express or the database, but on ports that the adapters implement. This makes testing easier and allows the persistence technology to be changed without touching the business logic.

The client is organized by features (feature/) with presentation, business, data, external services, and configuration layers. The details are found in the software design document.

Development methodology

The project is managed with Scrum (Schwaber & Sutherland, 2020) in two-week sprints. The instructor acts as product owner, who reviews and accepts each increment, and the technical lead also fulfills the role of Scrum Master.

Table 4
Scrum events and artifacts in the project

Element	Application in the project
Sprint	Two weeks; 14 sprints planned between January 1 and July 31, 2026
Planning	Selection of user stories from the backlog according to MoSCoW priority
Review	Demonstration of the increment to the instructor
Retrospective	Improvement actions recorded in the corrective actions document
Product backlog	User stories traced to the requirements (03-product/product-backlog.md)
Definition of done	Code reviewed, tests passing, accessibility verified, and documentation updated
Git workflow	Feature branch, commits in English with Conventional Commits, pull request with squash and merge

Schedule

Table 5
Schedule by phases

Phase	Sprints	Period	Deliverables
Analysis and requirements	1–2	January 2026	Requirements report, user stories, use cases
Design	3–4	February 2026	Architecture, data model, interface prototypes
Core construction	5–9	March to May 2026	Authentication, translation, alphabet, history
Complementary modules	10–12	June 2026	Profile, notifications, statistics, achievements
Testing and deployment	13–14	July 2026	Test plan and results, deployment, manuals
Restructuring into microservices	Re-planning	September to October 2026	Separation of the backend into six components and updated documentation

Note. The original baseline closed on July 31, 2026; the microservices phase corresponds to the subsequent re-planning.

Resources

Human resources

Table 6
Project team

Name	Role	Responsibility	Contact
Juan Mauricio Suaza Solórzano	Technical lead, Scrum Master, analyst and developer	Analysis, design, backend programming (lexicon and recognition), and coordination	Through the training programme
Luis Alejandro Duarte Aldana	Analyst and developer	Analysis, interface design, and programming	Through the training programme
Juan Camilo Fierro Esquivel	Analyst and developer	Analysis, product documentation, and programming	Through the training programme

Note. Apprentices from the Technologist in Software Analysis and Development program at SENA.

Technical resources

- Three development computers with at least 16 GB of RAM and Docker Desktop.
- Mid-range Android and iOS phones for camera and performance testing.
- Server (physical or cloud) with 4 vCPU, 8 GB of RAM, and 50 GB of disk for deployment.
- GitHub accounts, an SMTP mail server, and Google and Facebook developer accounts for external login.

Budget

Table 7
Estimated project budget (Colombian pesos)

Item	Detail	Value (COP)
Development and programming	Three developers for six months (lead: $3,000,000/month; analysts: $2,500,000/month each)	48 000 000
Hardware	Development computers and test mobile devices	15 003 310
Infrastructure and servers	Deployment server and domain	1 350 000
Testing and training	Pilot testing and user training	2 000 000
Software and security	Certificates and tools	225 000
Total		**66 578 310**

Note. The total corresponds to the sum of the components of the Technical and Economic Proposal; it is the figure considered valid while the partial totals circulating in other documents are reconciled.

Main technical risks

Table 8
Risks and mitigation strategy

Risk	Level	Mitigation
Insufficient LSC training data	Critical	Use the public LSC-54 dataset and capture samples with consent; retrain with the most failed signs
Recognition accuracy lower than expected	High	Threshold that prevents showing doubtful translations; per-sign metrics before publishing templates
Small team facing the complexity of microservices	High	Shared package (@traduce/shared), service templates, and Docker Compose
Privacy of samples (Ley 1581 de 2012)	High	Mandatory consent per sample, access only for ADMIN, video processed on the device
Single server without redundancy	Medium	Daily backups and documented restoration procedure

Deliverables

1. Requirements specification report.
2. Analysis documentation (dynamic views).
3. Technical proposal (this document).
4. Software design documentation.
5. Code artifacts: mobile, web, and desktop (via browser) applications and backend.
6. Testing documentation.
7. Implementation procedure.
8. Documentation of corrective, preventive, and improvement actions.
9. Technical, installation, and end-user manuals.
10. Project article.

Conclusions

The proposed solution is viable for a team of three apprentices because it relies on mature components (MediaPipe, Expo, PostgreSQL, RabbitMQ) and concentrates the team's own effort on LSC recognition and the accessible experience. The microservices architecture allows growth in parts, and on-device recognition meets the real-time requirement without exposing the user's video.

References

Broadcom. (2026). *RabbitMQ documentation*. https://www.rabbitmq.com/docs

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Congreso de la República de Colombia. (2012, October 17). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48,587.

Docker Inc. (2026). *Docker Compose documentation*. https://docs.docker.com/compose/

Expo. (2026). *Expo documentation*. https://docs.expo.dev/

Instituto Nacional para Sordos. (2020). *Observatorio social de la población sorda colombiana*. INSOR. https://www.insor.gov.co/observatorio/

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (2019). *MediaPipe: A framework for building perception pipelines* (arXiv:1906.08172). arXiv. https://doi.org/10.48550/arXiv.1906.08172

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.

World Health Organization. (2021). *Informe mundial sobre la audición*. https://www.who.int/publications/i/item/9789240020481

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). *MediaPipe Hands: On-device real-time hand tracking* (arXiv:2006.10214). arXiv. https://doi.org/10.48550/arXiv.2006.10214